SÉCURITÉ APPLICATIVE – TP

Secure Coding Best Practices

Audit & Sécurisation d'une Application PHP d'Authentification

Basé sur les recommandations **OWASP Secure Coding Practices (SCP)**

Étudiant : Talel Chaanbi

Module : Sécurité des Applications Web

Sujet : Révision et sécurisation d'un code PHP selon OWASP SCP

Date : 06 Mars 2026

CONFIDENTIEL – Usage académique uniquement

Table des Matières

1. Introduction & Contexte 3
2. Audit Initial – Vulnérabilités Identifiées 3
3. Bonne Pratique 1 – Hachage Sécurisé des Mots de Passe 4
4. Bonne Pratique 2 – Protection contre la Force Brute 4
5. Bonne Pratique 3 – Jetons CSRF 5
6. Bonne Pratique 4 – Gestion Sécurisée des Sessions 5
7. Bonne Pratique 5 – Sécurité des Cookies de Session 6
8. Bonne Pratique 6 – Gestion Sécurisée des Erreurs 6
9. Bonne Pratique 7 – Gestion des Credentials 7
10. Tableau Récapitulatif des Modifications 7
11. Conclusion & Recommandations 8

9	9	7	4
VULNÉRABILITÉS DÉTECTÉES	CORRECTIONS APPLIQUÉES	BONNES PRATIQUES OWASP	FICHIERS MODIFIÉS

📖 **Référence :** OWASP Secure Coding Practices – Quick Reference Guide v2.0.
Les numéros de section (§) référencés dans ce rapport correspondent aux chapitres du guide OWASP SCP.

1 Introduction & Contexte

Dans le cadre du TP de sécurité applicative, une application PHP d'authentification a été développée lors d'une activité précédente. Cette application comprend les fichiers suivants :

Fichier	Rôle
`index.php`	Page de connexion (formulaire login/mot de passe)
`config.php`	Configuration de la connexion à la base de données
`dashboard.php`	Page privée accessible après authentification
`logout.php`	Script de déconnexion et destruction de session

L'objectif de ce rapport est de présenter l'audit du code original, les vulnérabilités identifiées, et les modifications appliquées selon les recommandations de l'**OWASP Secure Coding Practices (SCP)**.

2 Audit Initial – Vulnérabilités Identifiées

🔴 Mot de passe en clair — CRITIQUE

Les mots de passe sont stockés et comparés en clair dans la base de données. Toute compromission de la DB expose immédiatement tous les comptes.

🔴 Absence de jeton CSRF — CRITIQUE

Aucune protection contre les attaques Cross-Site Request Forgery. Un attaquant peut forger une requête POST de connexion au nom d'un utilisateur.

🔴 Pas de protection force brute — CRITIQUE

Aucune limitation des tentatives de connexion. Un attaquant peut tester des milliers de mots de passe sans restriction.

🔴 Fixation de session — CRITIQUE

L'ID de session n'est pas régénéré après la connexion. Un attaquant qui connaît l'ID de session peut l'utiliser pour prendre le contrôle du compte.

🟡 Cookie session non sécurisé — MOYEN

Les flags `HttpOnly`, `SameSite` et `Secure` ne sont pas configurés, exposant le cookie à des vols via XSS ou inter-sites.

🟡 Pas de timeout de session — MOYEN

La session ne expire jamais par inactivité. Un poste laissé sans surveillance reste connecté indéfiniment.

🟡 Déconnexion incomplète — MOYEN

La déconnexion détruit la session côté serveur mais ne supprime pas le cookie côté client, permettant une réutilisation potentielle.

🟡 Credentials codés en dur — MOYEN

Les identifiants de la base de données sont écrits en clair dans le code source. Tout accès au dépôt Git ou au fichier expose les credentials.

🔴 Erreur DB exposée publiquement — CRITIQUE

En cas d'erreur de connexion à la base de données, le message d'erreur MySQL complet était affiché à l'utilisateur, révélant des informations sur l'infrastructure.

3 BP1 – Hachage Sécurisé des Mots de Passe

📌 **OWASP SCP §2 – Authentication :** « Store passwords using a strong, adaptive and salted hashing algorithm. Never store or transmit passwords in clear text. »

La vulnérabilité principale du code original était la comparaison des mots de passe en clair directement dans la requête SQL. Cette approche expose les mots de passe en cas de compromission de la base de données et permet des attaques par timing.

❌ **AVANT – Mot de passe en clair dans SQL**

```
$sql = 'SELECT login FROM users
  WHERE login = ?
  AND pass = ? LIMIT 1';

$stmt->bind_param('ss', $login, $password);
// Le mot de passe transite dans la requête SQL
// Stockage en clair dans la base de données
```

✅ **APRÈS – Hash bcrypt + password_verify()**

```
// Récupère seulement par login
$sql = 'SELECT login, pass FROM users
  WHERE login = ? LIMIT 1';
$stmt->bind_param('s', $login);

// Comparaison sécurisée en PHP
$valid = $user &&
  password_verify($password, $user['pass']);
```

Un script de migration **hash_password.php** a été créé pour convertir les mots de passe existants en hashes bcrypt via `password_hash($pass, PASSWORD_DEFAULT)`.

✅ **Résultat :** Les mots de passe sont stockés sous forme de hash bcrypt adaptatif (coût 12 par défaut). Même en cas de fuite de la base de données, les mots de passe originaux ne peuvent pas être retrouvés directement.

4 BP2 – Protection contre la Force Brute

📌 **OWASP SCP §2 – Authentication :** « Lock out accounts after a defined number of failed login attempts. »

Le code original n'imposait aucune limite sur les tentatives de connexion. Un attaquant pouvait tester des milliers de combinaisons sans aucune restriction.

✅ APRÈS – Verrouillage après 5 tentatives échouées (index.php)

```
define('MAX_FAILED_ATTEMPTS', 5);
define('LOCKOUT_DURATION',    300); // 5 minutes en secondes

// Vérification du verrouillage
if (
    isset($_SESSION['failed_attempts'], $_SESSION['lockout_time']) &&
    $_SESSION['failed_attempts'] >= MAX_FAILED_ATTEMPTS &&
    (time() - $_SESSION['lockout_time']) < LOCKOUT_DURATION
) {
    $remaining = LOCKOUT_DURATION - (time() - $_SESSION['lockout_time']);
    $message   = "Compte temporairement bloqué. Réessayez dans {$remaining} s.";
}

// Incrément du compteur en cas d'échec
$_SESSION['failed_attempts'] = ($_SESSION['failed_attempts'] ?? 0) + 1;
if ($_SESSION['failed_attempts'] >= MAX_FAILED_ATTEMPTS) {
    $_SESSION['lockout_time'] = time();
}
```

✅ **Résultat :** Après 5 tentatives échouées, le compte est verrouillé pendant 5 minutes. Le compteur est remis à zéro après une connexion réussie.

⚠️ **Note Production :** Pour une application en production, il est recommandé de stocker le compteur en base de données (par IP et par login) plutôt qu'en session, et d'implémenter un CAPTCHA après le 3e échec.

5 BP3 – Protection CSRF par Jeton Synchronisé

> 📌 **OWASP SCP §5 – CSRF Protection :** « Include a unique token in all forms and verify it on the server side. Use hash_equals() for timing-safe comparison. »

Une attaque CSRF (Cross-Site Request Forgery) permet à un attaquant de forger une requête HTTP au nom d'un utilisateur authentifié. Sans protection, une page malveillante peut soumettre le formulaire de connexion silencieusement.

❌ **AVANT – Aucune protection CSRF**

```
<form method="post" action="">
  <!-- Aucun jeton CSRF -->
  <input name="login" type="text">
  <input name="password" type="password">
  <button type="submit">Connexion</button>
</form>
```

✅ **APRÈS – Jeton CSRF synchronisé**

```
// Génération du jeton (PHP)
$_SESSION['csrf_token'] =
  bin2hex(random_bytes(32));

// Vérification (timing-safe)
if (!hash_equals(
  $_SESSION['csrf_token'],
  $_POST['csrf_token'] ?? ''
)) { /* rejet */ }

// Formulaire HTML
<input type="hidden" name="csrf_token"
  value="<?= $token ?>">
```

Le jeton est **renouvelé après chaque soumission** (réussie ou échouée), rendant les tokens capturés inutilisables pour de futures requêtes.

> ✅ **Résultat :** Chaque formulaire contient un jeton aléatoire de 256 bits lié à la session. `hash_equals()` protège contre les attaques de timing sur la comparaison de chaînes.

6 BP4 – Gestion Sécurisée des Sessions

> 📌 **OWASP SCP §3 – Session Management :** « Regenerate session ID after successful login. Set session inactivity timeout. »

6.1 – Régénération de l'ID de session (fixation de session)

❌ **AVANT – ID de session conservé**

```
if ($result->num_rows === 1) {
  $_SESSION['login'] = $login;
  header('Location: dashboard.php');
  exit;
  // L'ID de session reste identique
  // → attaque de fixation possible
```

✅ **APRÈS – Nouvel ID après connexion**

```
if ($valid) {
  // Génère un nouvel ID de session
  // et invalide l'ancien
  session_regenerate_id(true);

  $_SESSION['login']         = $login;
  $_SESSION['last_activity'] = time();
  header('Location: dashboard.php');
}
```

6.2 – Timeout d'inactivité (dashboard.php)

✅ APRÈS – Expiration après 15 minutes d'inactivité (dashboard.php)

```
define('SESSION_TIMEOUT', 900); // 15 minutes

if (isset($_SESSION['last_activity']) &&
    (time() - $_SESSION['last_activity']) > SESSION_TIMEOUT) {
    session_unset();
    session_destroy();
    header('Location: index.php?timeout=1');
    exit;
}
// Mise à jour du timestamp à chaque requête
$_SESSION['last_activity'] = time();
```

✅ **Résultat :** L'ID de session est régénéré à la connexion (prévention fixation). La session expire automatiquement après 15 minutes d'inactivité.

7 BP5 – Sécurité des Cookies de Session

📌 **OWASP SCP §3 – Session Management :** « Set HttpOnly, SameSite=Strict, and Secure flags on session cookies. »

❌ **AVANT – Aucun flag de sécurité**

```
// Dans config.php : aucune configuration
// des flags du cookie de session.
// Le cookie est accessible en JavaScript,
// envoyé sur des requêtes inter-sites,
// et accepte les IDs fixés par l'attaquant.
```

✅ **APRÈS – Flags sécurisés (config.php)**

```
// HttpOnly : cookie inaccessible en JS
ini_set('session.cookie_httponly',  1);
// SameSite=Strict : bloque inter-sites
ini_set('session.cookie_samesite', 'Strict');
// use_strict_mode : rejette IDs non-serveur
ini_set('session.use_strict_mode',  1);
// Secure : HTTPS uniquement (production)
ini_set('session.cookie_secure', 0); // → 1 en prod
```

Terminaison complète de la session – logout.php

❌ **AVANT – Cookie non supprimé côté client**

```
session_start();
session_unset();
session_destroy();
// Le cookie PHPSESSID reste dans
// le navigateur → réutilisable !
```

✅ **APRÈS – Cookie expiré côté client**

```
session_unset();
if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '',
        time() - 42000,  // passé → supprimé
        $p['path'], $p['domain'],
        $p['secure'], $p['httponly']);
}
session_destroy();
```

✅ **Résultat :** Le cookie de session est définitivement supprimé côté client lors de la déconnexion.

8 BP6 – Gestion Sécurisée des Erreurs

📌 **OWASP SCP §7 – Error Handling :** « Do not disclose sensitive information in error responses. Log errors server-side only. »

❌ **AVANT – Erreur MySQL affichée publiquement**

```
die('Erreur connexion MySQL : '
    . $conn->connect_error);
// Révèle : version MySQL, user, hôte...
```

✅ **APRÈS – Journalisation + message générique**

```
error_log('Erreur MySQL : '
    . $conn->connect_error); // côté serveur
http_response_code(500);
die('Service temporairement indisponible.');
```

De même, les erreurs de préparation de requêtes SQL et l'erreur d'authentification retournent un message générique : « login ou mot de passe invalide » sans préciser lequel des deux est erroné.

9 BP7 – Gestion Sécurisée des Credentials

📌 **OWASP SCP §2 :** « Do not hard-code credentials. Use environment variables or external secret management. »

❌ **AVANT – Credentials en dur dans le code**

```
$DB_HOST = 'localhost';
$DB_USER = 'root';
$DB_PASS = '123456'; // visible dans Git !
$DB_NAME = 'authdb';
```

✅ **APRÈS – Variables d'environnement**

```
$DB_HOST = getenv('DB_HOST') ?: 'localhost';
$DB_USER = getenv('DB_USER') ?: 'root';
$DB_PASS = getenv('DB_PASS') ?: '123456';
$DB_NAME = getenv('DB_NAME') ?: 'authdb';
// Définir DB_PASS dans l'env du serveur
```

✅ **Résultat :** En production, les credentials sont injectés via des variables d'environnement et n'apparaissent plus dans le code source ni dans l'historique Git.

10 Tableau Récapitulatif des Modifications

#	Fichier	Vulnérabilité initiale	Correction appliquée	OWASP SCP	Sévérité
1	`index.php`	Mot de passe comparé en clair dans SQL	`password_hash()` + `password_verify()`	§2	Critique
2	`index.php`	Aucune protection force brute	Verrouillage après 5 échecs, blocage 5 min	§2	Critique
3	`index.php`	Absence de jeton CSRF	Token `bin2hex(random_bytes(32))` + `hash_equals()`	§5	Critique
4	`index.php`	ID de session non régénéré	`session_regenerate_id(true)` post-login	§3	Critique
5	`index.php`	Message d'erreur différencié (login vs pass)	Message générique identique	§2	Moyen
6	`dashboard.php`	Pas de timeout de session	Expiration après 15 min d'inactivité	§3	Moyen
7	`logout.php`	Cookie non supprimé côté client	`setcookie()` avec date passée	§3	Moyen
8	`config.php`	Flags de session absents (HttpOnly, SameSite)	`ini_set()` pour HttpOnly + SameSite=Strict + Strict Mode	§3	Moyen
9	`config.php`	Erreur MySQL exposée + credentials en dur	`error_log()` + `getenv()`	§2 §7	Moyen

Checklist de conformité OWASP SCP

- ✅ Mots de passe hashés avec bcrypt (PASSWORD_DEFAULT) – jamais en clair
- ✅ Requêtes préparées (prepared statements) – protection SQL Injection
- ✅ Jeton CSRF renouvelé à chaque soumission de formulaire
- ✅ Session régénérée après chaque connexion réussie
- ✅ Timeout d'inactivité de 15 minutes implémenté
- ✅ Verrouillage de compte après 5 tentatives échouées
- ✅ Cookie HttpOnly + SameSite=Strict configuré
- ✅ Messages d'erreur génériques côté client
- ✅ Erreurs techniques journalisées côté serveur uniquement
- ⚠️ Credentials via variables d'environnement (fallback en dur en développement)
- ⚠️ HTTPS + cookie Secure : à activer en production
- ⚠️ Compteur de force brute en session (à migrer en DB en production)
- ❌ CAPTCHA : non implémenté (recommandé pour la production)

❌ Authentification multi-facteur (MFA) : non implémentée

11 Conclusion & Recommandations

Ce TP a permis de mettre en pratique les principales recommandations de l'**OWASP Secure Coding Practices** sur une application PHP d'authentification réelle. Le code initial présentait des vulnérabilités classiques et critiques, fréquemment rencontrées dans des applications web développées sans considération de sécurité.

🔐 Synthèse des améliorations

✅ **9 vulnérabilités corrigées** dans 4 fichiers PHP
✅ **7 bonnes pratiques OWASP SCP** appliquées (§2, §3, §5, §7)
✅ Niveau de sécurité **significativement amélioré** par rapport au code original
✅ Chaque modification est **documentée et référencée** au guide OWASP

Recommandations pour la mise en production

Priorité	Recommandation	Référence
Critique	Activer HTTPS et passer `cookie_secure = 1`	OWASP SCP §3
Critique	Supprimer `hash_password.php` après la migration	OWASP SCP §2
Critique	Stocker les credentials DB dans des variables d'environnement serveur	OWASP SCP §2
Élevé	Déplacer `config.php` en dehors de la racine web	OWASP SCP §7
Élevé	Migrer le compteur de force brute vers la base de données	OWASP SCP §2
Élevé	Ajouter un CAPTCHA après 3 tentatives échouées	OWASP SCP §2
Moyen	Mettre en place une authentification multi-facteur (MFA/TOTP)	OWASP SCP §2
Moyen	Configurer un en-tête `Content-Security-Policy`	OWASP SCP §5
Moyen	Journalisation des tentatives de connexion avec IP et horodatage	OWASP SCP §8

📚 **Références :**

• OWASP Secure Coding Practices – Quick Reference Guide v2.0 : owasp.org/www-project-secure-coding-practices-quick-reference-guide
• PHP Manual – `password_hash()`, `session_regenerate_id()`, `random_bytes()`
• OWASP CSRF Prevention Cheat Sheet : cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet